PROGRAMMING & CS FUNDAMENTALS

Garbage Collection Visual Guide

Automatic memory reclamation — how runtimes find what's dead.

ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Why GC Exists

Manual freeing is correct until it isn't — at scale, humans lose.

manual

fast, precise■+ leaks/corruption

GC

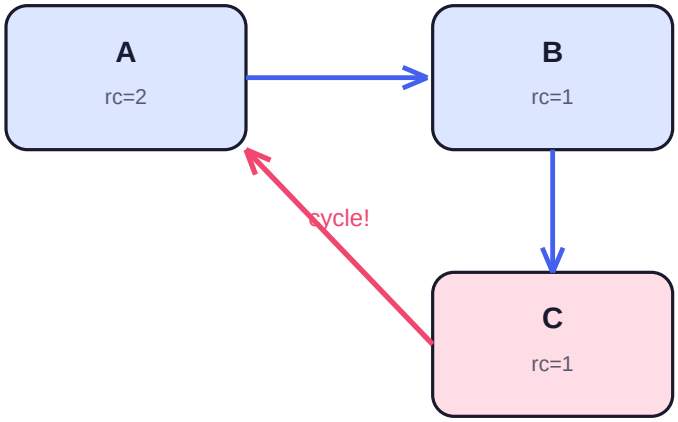
safe by default■+ pause/throughput cost

- GC trades CPU cycles and occasional pauses for memory safety.
- Roots: globals, stacks, registers — everything reachable FROM roots is alive.
- Unreachable = garbage, even if pointers still dangle between dead objects.
- No GC prevents LOGICAL leaks: a global list growing forever is still your bug.

STEP 2/6

Reference Counting

Count incoming pointers; zero means dead — immediate, no pauses.



B <-> C reference each other: rc never hits 0 -> LEAKED forever

```
# CPython: refcounting + a cycle detector bolted on
import gc; gc.collect()   # hunt the cycles refcounting can't see
```

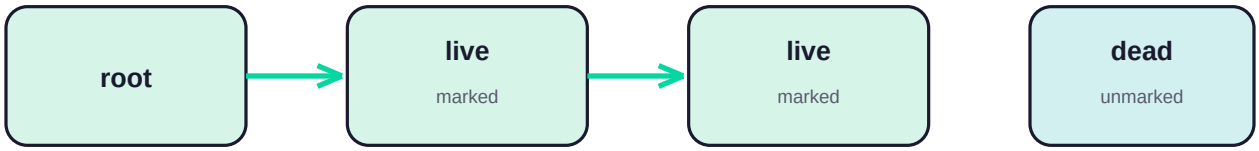
- Pros: prompt freeing, no stop-the-world pauses, simple.
- Cons: cycles leak; every pointer write pays counter churn (cache traffic).
- Weak references break cycles: caches and parent-links should be weak.

STEP 3/6

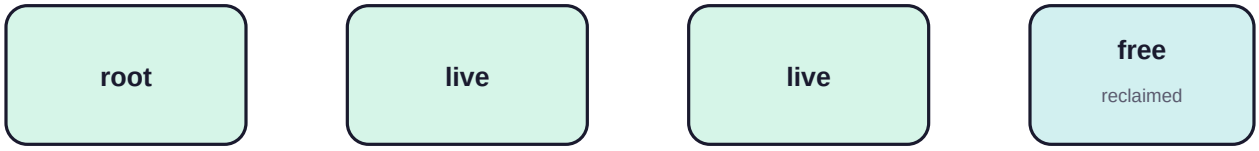
Mark-and-Sweep

Trace from roots, then sweep the unmarked — the classic two-phase GC.

Phase 1 — MARK (trace from roots):



Phase 2 — SWEEP (reclaim unmarked):



- Handles cycles naturally: unreachable cycles are simply never marked.
- Cost: the whole heap is traced — pause scales with LIVE data, not garbage.
- Fragmentation: sweeping leaves holes — compacting variants relocate survivors.

STEP 4/6

Generational GC: The Age Trick

Most objects die young — collect the nursery often, the elders rarely.

Minor GC: scan young gen only (fast, frequent). Survivors get promoted.

Major GC: scan everything (slow, rare) — the pause users feel.

- Generational hypothesis: 90%+ of objects die in the nursery — bet the GC on it.
- Write barriers track old-to-young pointers so minor GCs stay correct.
- Tuning = sizing generations: too-small nursery promotes garbage; too-big pauses.

STEP 5/6

Pauses: The Latency Battle

Throughput vs latency — pick your GC by your SLA.

Serial / Parallel

stop-the-world, max throughput — batch jobs

CMS / G1

mostly concurrent, short pauses — web services

ZGC / Shenandoah

sub-ms pauses, colored pointers — trading, gaming

No GC (Rust/ARC)

zero pauses, compile-time — systems, real-time

- Stop-the-world: every mutator thread halts — simple, but the tail latency spikes.
- Concurrent GC trades throughput (barrier overhead) for shorter pauses.
- Measure p99, not average: GC pauses live in the tail.

STEP 6/6

GC-Friendly Code

Write so the collector barely notices you.

- Short-lived garbage is FREE in generational GCs — don't pool tiny objects prematurely.
- Long-lived caches are expensive: they tenured and get scanned every major GC — bound them.
- Avoid finalizers: they resurrect objects and serialize collection — use explicit close().
- Null out (or scope-limit) references you no longer need in long-lived structures.
- Allocation rate drives GC frequency: fewer temporaries, fewer collections.
- Know your flags: -Xmx, G1 regions, ZGC generations — defaults are for average apps, not yours.

STEP ✓I✓

Key Takeaways

Garbage Collection — Remember

- Roots (stacks, globals) define liveness: unreachable objects are garbage, cycles included.
- Reference counting frees promptly but leaks cycles and churns on every pointer write.
- Mark-and-sweep traces from roots then reclaims unmarked — pause scales with live data.
- Generational GC exploits 'most die young': frequent minor collections, rare major ones.
- Pick collectors by SLA (throughput vs p99 pauses); short-lived garbage is nearly free.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev